

CV6: Refaktoring používateľského rozhrania pomocou Material3 - Teória

1 Cieľ cvičenia

V tomto cvičení sa budete venovať tomu, ako zlepšovať existujúce používateľské rozhranie pomocou princípov Material Design 3. Na rozdiel od predchádzajúcich tém už nepôjde iba o vytvorenie ďalšej Compose obrazovky, ale o premyslenú úpravu rozhrania tak, aby bolo konzistentné, prehľadné a vizuálne zjednotené.

Po absolvovaní cvičenia by ste mali rozumieť tomu, prečo nestačí, aby aplikácia iba fungovala, ale prečo je dôležité, aby používala vhodné komponenty, správnu typografiu, jednotnú farebnú schému a zrozumiteľnú hierarchiu obsahu. Zároveň by ste mali vedieť vysvetliť, akú úlohu zohráva Material3 v Jetpack Compose a prečo je vhodné používať tému aplikácie namiesto explicitne zapisovaných vizuálnych hodnôt.

Toto cvičenie zároveň predstavuje prechod od základného skladania UI k vedomejšiemu navrhovaniu rozhrania. V doterajších cvičeniach bolo prioritou najmä to, aby obrazovka fungovala a vedela reagovať na vstupy alebo stav. V tomto cvičení sa dôraz presúva na kvalitu výsledného rozhrania. Budete teda premýšľať nielen nad tým, čo sa má zobrazíť, ale aj nad tým, ako sa to má zobrazíť a prečo je dané riešenie vhodné.

2 Prečo má zmysel venovať sa Material3 až teraz

Na prvý pohľad sa môže zdať, že Material komponenty ste už používali aj v skorších cvičeniach, a teda nejde o novú tému. To je pravda iba čiastočne. V predchádzajúcich cvičeniach boli Material prvky skôr nástrojom, pomocou ktorého ste vytvorili formulár, tlačidlo alebo zoznam. V tomto cvičení sa však Material3 stáva samotnou témou. Nejde iba o to, že použijete `Button`, ale o to, že začnete uvažovať nad rozhraním ako nad systémom.

Práve tu sa objavuje rozdiel medzi funkčným a kvalitným UI. Dve obrazovky môžu zobrazovať rovnaké dáta a poskytovať rovnaké akcie, ale iba jedna z nich môže pôsobiť čitateľne, moderne a konzistentne. Material3 pomáha dosiahnuť túto konzistentnosť tým, že definuje spoločné pravidlá pre farby, texty, rozloženie a komponenty.

3 Čo je Material Design 3

Material Design 3 je dizajnový systém vytvorený spoločnosťou Google. Jeho cieľom je poskytnúť jednotné pravidlá pre tvorbu moderných používateľských rozhraní. V prostredí Jetpack Compose je Material3 dostupný ako sada hotových komponentov a zároveň ako systém pravidiel, ktoré určujú spôsob práce s farbami, typografiou a tvarmi.

Je dôležité uvedomiť si, že Material3 nie je iba zbierka tlačidiel a kariet. Predstavuje širší návrhový systém, ktorý má zabezpečiť konzistentný vzhľad a správanie celej aplikácie. Keď používate Material3 správne, jednotlivé komponenty spolu vizuálne súvisia a aplikácia pôsobí ako jeden ucelený celok. To znamená, že namiesto náhodného rozhodovania o tom, aké farby, písmo alebo zaoblenie použijete, sa opierate o vopred definovaný systém. Výsledkom je používateľské rozhranie, ktoré je jednoduchšie na údržbu, lepšie čitateľné a spravidla aj profesionálnejšie pôsobiace.

4 Material3 v Jetpack Compose

V Jetpack Compose sa Material3 najčastejšie používa prostredníctvom knižnice `androidx.compose.material3`. Táto knižnica obsahuje základné komponenty ako tlačidlá, karty, horné aplikačné panely, textové polia, prepínače, čipy a ďalšie prvky.

Príklady bežných Material3 komponentov:

- `Button`
- `OutlinedButton`
- `Card`
- `Surface`
- `TopAppBar`
- `TextField`
- `FilterChip`

Používanie týchto komponentov má výhodu v tom, že už samy o sebe rešpektujú základné návrhové pravidlá Material3. Ak ich navyše prepojíte s témou aplikácie, získate používateľské rozhranie, ktoré je konzistentné a jednoduchšie na údržbu.

Treba si všimnúť aj to, že Material3 v Compose nie je niečo „nad Compose“, ale je to prirodzená súčasť spôsobu, akým Compose obrazovky vznikajú. Komponenty, ktoré skladáte, sú stále composable funkcie. Rozdiel je v tom, že namiesto čisto neutrálnych layout kontajnerov pracujete s komponentmi, ktoré majú dizajnový význam.

5 MaterialTheme ako základ dizajnového systému

Najdôležitejším prvkom Material3 v Compose je `MaterialTheme`. Ide o centrálny objekt, ktorý definuje vizuálnu identitu aplikácie. Obsahuje tri hlavné oblasti: farebnú schému, typografiu a tvary komponentov.

- `colorScheme` určuje farby používané v aplikácii
- `typography` určuje štýly textu
- `shapes` určuje zaoblenie a tvar komponentov

Ak aplikácia používa `MaterialTheme`, komponenty môžu automaticky čerpať farby a štýly z tejto centrálnej definície. To znamená, že pri zmene témy sa nemusí upravovať každá obrazovka osobitne.

```
MaterialTheme(  
    colorScheme = lightColorScheme(),  
    typography = Typography(),  
    shapes = Shapes()  
) {  
    AppContent()  
}
```

Táto myšlienka je dôležitá najmä pri refaktoringu. Ak totiž pôvodná obrazovka používa pevne zapísané farby a veľkosti písma, veľmi ťažko sa neskôr upravuje. Ak však tie isté vizuálne rozhodnutia presuniete do témy, získate aplikáciu, v ktorej sa zmeny robia omnoho systematickejšie.

6 Farebná schéma

Farebná schéma v Material3 slúži na definovanie rolí farieb v používateľskom rozhraní. Namiesto toho, aby ste používali ľubovoľné farby priamo v komponentoch, je vhodné pracovať s rolami ako `primary`, `secondary`, `surface`, `background` alebo `error`.

Tento prístup je dôležitý preto, že farba potom nevyjadruje iba konkrétnu vizuálnu hodnotu, ale aj svoj význam v rozhraní. Napríklad farba `primary` sa zvyčajne používa na zvýraznenie hlavných akcií, zatiaľ čo `surface` sa často používa pre karty alebo panely.

```
val cardColor = MaterialTheme.colorScheme.surface  
val buttonColor = MaterialTheme.colorScheme.primary
```

Používaním farebnej schémy získavate väčšiu flexibilitu a zároveň podporujete konzistentnosť celej aplikácie.

Pri refaktoringu je veľmi častou chybou, že sa budete snažiť „zachrániť“ nevhodné rozhranie iba výmenou jednej sivej farby za inú sivú farbu. Takýto prístup však problém nerieši, iba posúva na inú úroveň. Oveľa správnejšie je premýšľať nad tým, akú úlohu má daný prvok na obrazovke. Je to hlavná akcia? Potom by mal pravdepodobne používať farby odvodené od `primary`. Je to karta s obsahom? Potom je prirodzené, že bude vychádzať z `surface`. Je to chybové hlásenie? Potom má zmysel použiť `error`.

7 Typografia

Typografia v Material3 určuje, ako majú vyzeráť textové prvky na rôznych úrovniach hierarchie. Namiesto explicitného nastavovania hodnôt ako `fontSize = 18.sp` je vhodnejšie používať preddefinované štýly z `MaterialTheme.typography`.

Medzi často používané štýly patria napríklad:

- `headlineSmall`
- `titleMedium`
- `bodyMedium`
- `labelLarge`

Vďaka týmto štýlom je rozhranie čitateľnejšie a texty sú medzi sebou vizuálne zladené. Ak každý nadpis a každý doplnkový text použije náhodnú veľkosť písma, rozhranie pôsobí nejednotne. Typografický systém tento problém rieši.

```
Text(  
    text = "Dashboard",  
    style = MaterialTheme.typography.headlineSmall  
)
```

Je užitočné vnímať typografiu ako systém rolí, nie ako systém čísel. Keď napíšete `fontSize = 24.sp`, hovoríte iba to, aké veľké má byť písmo. Keď použijete `headlineSmall`, hovoríte zároveň aj to, akú úlohu daný text zohráva v hierarchii obrazovky. Tento rozdiel je pri návrhu UI veľmi dôležitý.

8 Shapes

Material3 pracuje aj s tvarmi komponentov. Tie sú definované v časti **shapes** a ovplyvňujú napríklad to, ako veľmi budú zaoblené rohy kariet alebo tlačidiel.

Aj keď sa tvary môžu na prvý pohľad zdať ako menej dôležitý detail, majú významný vplyv na výsledný charakter rozhrania. Jednotné shapes prispievajú k tomu, aby všetky komponenty pôsobili ako súčasť jedného systému.

Pri refaktoringu sa shapes často nevnímajú ako prvá priorita, no je dobré vedieť, že aj nekonzistentné zaoblenia môžu znižovať vizuálnu kvalitu obrazovky. Ak má každá karta iné rohy a tlačidlá pôsobia odlišne od zvyšku rozhrania, aplikácia stratí jednotný charakter.

9 Card a Surface

V tomto cvičení pri refaktoringu používateľského rozhrania zohrávajú dôležitú úlohu komponenty **Card** a **Surface**. Oba slúžia ako kontajnery, ale používajú sa v trochu odlišných situáciách.

Surface je univerzálny kontajner, ktorý umožňuje aplikovať farbu, tvar alebo ďalšie vizuálne vlastnosti. **Card** je špecializovanejší komponent, ktorý sa často používa na oddelenie logických blokov obsahu, napríklad jednotlivých sekcií dashboardu alebo položiek zoznamu.

```

Card(
    modifier = Modifier.fillMaxWidth()
) {
    Column(
        modifier = Modifier.padding(16.dp)
    ) {
        Text("Study progress")
        Text("You have completed 3 of 5 tasks.")
    }
}

```

Používanie týchto kontajnerov je vhodnejšie než všeobecný `Box`, ak chcete vytvoriť obrazovku, ktorá rešpektuje princípy Material Designu. To avšak neznamená že `Box` je nesprávny komponent. Ide o veľmi užitočný základný layout prvok. Problém nastáva vtedy, keď je používaný ako univerzálne riešenie na všetko. Ak má blok obsahu predstavovať samostatnú vizuálnu sekciu obrazovky, `Card` alebo `Surface` zvyčajne lepšie vyjadrujú zámer návrhu.

10 Modifier

`Modifier` je základný nástroj na úpravu vzhľadu a správania komponentov v `Compose`. Pomocou `Modifiera` môžete definovať veľkosť komponentu, jeho odsadenie, zarovnanie, pozadie alebo reakciu na kliknutie.

Medzi najčastejšie používané operácie patria:

- `padding()`
- `fillMaxWidth()`
- `size()`
- `background()`
- `clickable()`

Dôležité je, že `Modifier` sa reťazí, a preto záleží aj na poradí jednotlivých operácií. Správne používanie `Modifiera` patrí medzi najdôležitejšie praktické zručnosti pri tvorbe `Compose` obrazoviek.

```

modifier = Modifier
    .fillMaxWidth()
    .padding(16.dp)

```

Pri tomto cvičení je zvlášť dôležité rozlišovať medzi odsadením obsahu vo vnútri komponentu a odsadením medzi komponentmi navzájom. Programátori často používajú `padding` intuitívne, ale nie vždy systematicky. Výsledkom potom bývajú obrazovky, kde sú niektoré medzery príliš malé, iné príliš veľké a celková kompozícia pôsobí nevyrovnane. Refaktoring UI je dobrá príležitosť naučiť sa používať `Modifier` cielene.

11 Ako rozpoznať nekonzistentné UI

Aby ste vedeli existujúce rozhranie opraviť, nestačí poznať jednotlivé komponenty. Musíte vedieť rozpoznať, že rozhranie je nekonzistentné. To je jedna z najdôležitejších praktických zručností v tomto cvičení.

Za nekonzistentné UI možno považovať napríklad také rozhranie, v ktorom sa bez jasného dôvodu miešajú rôzne veľkosti písma, používajú sa náhodné farby, chýba vizuálne oddelenie sekcií a odsadenie medzi prvkami nepôsobí jednotne. Podobne problematické je aj to, keď nie je zrejmé, ktorý text je hlavný nadpis, ktorý text iba dopĺňa informáciu a ktoré tlačidlo predstavuje hlavnú akciu.

Typickým príkladom je obrazovka, kde sa všetko skladá iba z `Column`, `Box` a `Text`, pričom farby sú definované natvrdo, veľkosti písma sa nastavujú ručne a jednotlivé sekcie sa oddeľujú náhodnými hodnotami `padding` alebo `Spacer`. Takéto riešenie môže fungovať, ale chýba mu systém.

12 Refaktoring UI ako proces

Refaktoring používateľského rozhrania neznamena, že obrazovku prepisujete od nuly bez plánu. Oveľa vhodnejšie je postupovať systematicky a riešiť problémy v logickom poradí.

Najprv je vhodné pozrieť sa na štruktúru obrazovky. Ak sú sekcie obsahu zobrazené iba ako obyčajné bloky bez vizuálneho oddelenia, je dobré uvažovať nad použitím `Card` alebo `Surface`. Potom má zmysel upraviť typografiu, aby bolo jasné, čo je hlavný nadpis, čo názov sekcie a čo doplnkový text. Až v ďalšom kroku sa oplatí doladiť farby a odsadenie.

13 Prepojenie teórie so samotným zadaním

V tomto cvičení dostanete už existujúcu obrazovku. Vašou úlohou nebude vymyslieť novú funkcionality, ale zlepšiť existujúcu prezentáciu obsahu. Aby bol prechod z teórie na zadanie zrozumiteľný, je dobré pozrieť sa na niekoľko typických premien.

Ak je v pôvodnom kóde použitý `Box` len ako sivý blok pre obsah, môže byť vhodné nahradiť ho komponentom `Card`. Ak je text definovaný ako `fontSize = 24.sp`, je vhodné zamyslieť sa, či by nebolo lepšie použiť `MaterialTheme.typography.headlineSmall`. Ak je farba tlačidla alebo pozadia definovaná explicitne, je prirodzené nahradiť ju hodnotou z `MaterialTheme.colorScheme`. Ak sú medzi prvkami náhodné medzery, je potrebné ich zjednotiť.

Tieto kroky sa môžu zdať jednoduché, no spolu vytvárajú zásadnú zmenu v kvalite výsledného rozhrania.

14 Krátke porovnanie pred a po

Nasledujúca ukážka ilustruje typický rozdiel medzi improvizovaným riešením a riešením založeným na `Material3`.

14.1 Pôvodný prístup

```
Box(  
    modifier = Modifier  
        .fillMaxWidth()  
        .background(Color.LightGray)  
        .padding(8.dp)  
) {  
    Column {  
        Text("Tasks today", fontSize = 18.sp)  
        Text("5 tasks pending", fontSize = 14.sp)  
        Button(onClick = {}) {  
            Text("Open")  
        }  
    }  
}
```

V tomto prístupe je viditeľné, že farba je definovaná natvrdo a typografia nepoužíva systém Material3. Navyše kontajner Box síce funguje, ale nevyjadruje dizajnový význam sekcie.

14.2 Refaktorovaný prístup

```
Card(  
    modifier = Modifier.fillMaxWidth()  
) {  
    Column(  
        modifier = Modifier.padding(16.dp)  
    ) {  
        Text(  
            text = "Tasks today",  
            style = MaterialTheme.typography.titleMedium  
        )  
        Text(  
            text = "5 tasks pending",  
            style = MaterialTheme.typography.bodyMedium  
        )  
        Button(onClick = {}) {  
            Text("Open")  
        }  
    }  
}
```

V druhej verzii je štruktúra obrazovky zrozumiteľnejšia, texty sú hierarchicky usporiadané a kontajner lepšie zodpovedá významu sekcie. Už táto malá zmena ukazuje, prečo má refaktoring používateľského rozhrania zmysel.

15 Prečo nestačí, aby UI iba fungovalo

Začiatočník pri vývoji UI často prirodzene premýšľa tak, že ak sa text zobrazí a tlačidlo reaguje na kliknutie, úloha je hotová. Z pohľadu funkcionality je to síce pravda, ale z pohľadu kvality používateľského rozhrania to nestačí.

Používateľ pri práci s aplikáciou nečíta rozhranie po riadkoch kódu. Vníma ho ako celok. Ak je obrazovka nekonzistentná, používateľ nemusí vedieť rýchlo rozlíšiť, čo je dôležité, čo je doplnková informácia a kde sa nachádza hlavná akcia. Nekvalitné UI preto nezhoršuje iba estetiku, ale aj orientáciu a použiteľnosť. Učíte sa robiť používateľské rozhranie prehľadnejším a zrozumiteľnejším.

16 Vzťah k stavu obrazovky

Hoci hlavnou témou cvičenia je Material3 a refaktoring UI, je vhodné všimnúť si aj súvislosť so stavom obrazovky. V Compose totiž to, čo sa zobrazuje, závisí od aktuálneho stavu. To platí nielen pre dáta, ale aj pre rôzne podoby používateľského rozhrania.

Ak napríklad obrazovka zobrazuje stav načítavania, stav chyby, prázdny stav alebo úspešný stav s dátami, vždy ide o ten istý princíp. Stav určuje, ktoré composable sa majú zobrazíť. Toto pozorovanie je dôležité aj preto, že v ďalších cvičeniach sa k nemu vrátite pri práci s ViewModelom.

V rámci tohto cvičenia môžu byť tieto stavy chápané ako bonusové rozšírenie. Už pri samotnom refaktoringu sa však oplatí myslieť na to, že kvalitná obrazovka by mala vedieť reagovať aj na menej ideálne situácie, nielen na scenár, keď sú dáta úspešne načítané.

17 Ako postupovať pri riešení zadania

Pri praktickom riešení úlohy môžete zvoliť jednoduchý metodický postup. Najprv si prezrite pôvodnú obrazovku a skúste si spísať, čo Vám na nej prekáža. V tejto fáze sa nesnažte hneď prepisovať kód. Dôležitejšie je naučiť sa rozpoznať problém.

V druhej fáze upravte základnú štruktúru. Zamyslite sa, ktoré časti obrazovky majú predstavovať samostatné bloky obsahu a ktoré komponenty sú na to najvhodnejšie. Potom upravte textové štýly tak, aby vznikla jasná hierarchia medzi nadpisom, doplnkovým textom a akciou.

Až následne prejdite na farebnú schému a padding. Práve v tejto fáze zvyčajne vzniká najväčší rozdiel medzi pôvodnou a refaktorovanou verziou. Nakoniec skontrolujte, či všetky prvky pôsobia ako súčasť jedného systému a či niektorá časť obrazovky nepôsobí vizuálne „mimo“.

18 Ikony a ich praktické používanie

Pri práci s ikonami v Jetpack Compose je potrebné dávať pozor na to, že nie všetky ikony sú dostupné v každom projekte automaticky. Základná sada ikon býva menšia a niektoré rozšírené ikony si vyžadujú doplnenie príslušnej závislosti.

Ak by sa Vám stalo, že niektorá ikona nejde importovať, nejde o chybu Compose ako takého, ale zvyčajne o rozdiel medzi základnou a rozšírenou ikonovou sadou. Pri tvorbe výučbových príkladov je preto vhodné buď používať iba overené základné ikony, alebo sa ikonám úplne vyhnúť, ak nie sú jadrom cvičenia.

V rámci tohto cvičenia je dôležité, aby ikony neodvádzali pozornosť od hlavnej témy. V tomto cvičení nimi možno obrazovku doplniť, ale nemali by byť nosným prvkom riešenia.

19 Užitočné zdroje

Tieto zdroje môžete využiť na rozšírenie svojich vedomostí o Material Design 3. Nemusíte ich prejsť celé na to, aby ste zvládli urobiť zadanie. Oficiálna dokumentácia Material3 pre Jetpack Compose:

<https://developer.android.com/develop/ui/compose/designsystems/material3>

Prehľad Compose komponentov:

<https://developer.android.com/develop/ui/compose/components>

Dokumentácia o stave v Jetpack Compose:

<https://developer.android.com/develop/ui/compose/state>

Material Design 3 guidelines:

<https://m3.material.io/>

Material komponenty a vzory:

<https://m3.material.io/components>

Material Symbols a ikony:

<https://fonts.google.com/icons>

Compose samples:

<https://github.com/android/compose-samples>

20 Zhrnutie

Po preštudovaní tejto časti by ste mali rozumieť tomu, že Material3 nie je iba knižnica jednotlivých komponentov, ale celý dizajnový systém. Mali by ste vedieť, akú úlohu má **MaterialTheme**, farebná schéma, typografia, shapes a Modifier. Zároveň by ste mali chápať, ako rozpoznať nekonzistentné používateľské rozhranie a akým spôsobom ho systematicky zlepšovať.

Dôležité je aj to, že toto cvičenie nemení funkčný obsah obrazovky, ale jej kvalitu. Práve v tom spočíva jeho hlavný prínos. Učí Vás premýšľať nad používateľským rozhraním nielen ako nad technickým výstupom, ale aj ako nad navrhovaným systémom.

Tieto poznatky budú dôležité aj v ďalších cvičeniach, kde sa k nim pridá navigácia medzi obrazovkami, ViewModel a neskôr aj perzistencia stavu alebo používateľských preferencií.